

C# Statistics Program

```
using System;
using System.Linq;
using System.Collections.Generic;

class StatisticsProgram
{
    static void Main()
    {
        double[] data = {
            115, 182, 191, 31, 196, 1099, 5, 172, 10, 179,
            83, 21, 20, 21, 186, 177, 195, 193, 188, 199,
            62, 109, 105, 183, 110
        };

        Array.Sort(data);

        double mean = Mean(data);
        double median = Median(data);
        double mode = Mode(data);
        double variance = Variance(data, mean);
        double stdDev = Math.Sqrt(variance);

        double q1 = Percentile(data, 25);
        double q2 = Percentile(data, 50);
        double q3 = Percentile(data, 75);

        double p20 = Percentile(data, 20);
        double p50 = Percentile(data, 50);

        double range = data.Max() - data.Min();
        double iqr = q3 - q1;

        double sumDeviation = data.Sum(x => x - mean);

        Console.WriteLine("Mean = " + mean);
        Console.WriteLine("Mode = " + mode);
        Console.WriteLine("Median = " + median);
        Console.WriteLine("Variance = " + variance);
        Console.WriteLine("Standard Deviation = " + stdDev);

        Console.WriteLine("P20 = " + p20);
        Console.WriteLine("P50 = " + p50);

        Console.WriteLine("Q1 = " + q1);
        Console.WriteLine("Q2 = " + q2);
        Console.WriteLine("Q3 = " + q3);

        Console.WriteLine("Range = " + range);
        Console.WriteLine("IQR = " + iqr);

        Console.WriteLine("Summation of Deviations = " + sumDeviation);
    }

    static double Mean(double[] data)
    {
        return data.Average();
    }

    static double Median(double[] data)
    {
        int n = data.Length;
        if (n % 2 == 0)
            return (data[n / 2 - 1] + data[n / 2]) / 2.0;
        else
            return data[n / 2];
    }

    static double Mode(double[] data)
```

```

    {
        return data.GroupBy(x => x)
            .OrderByDescending(g => g.Count())
            .First().Key;
    }

    static double Variance(double[] data, double mean)
    {
        double sum = 0;
        foreach (var x in data)
            sum += Math.Pow(x - mean, 2);

        return sum / data.Length;
    }

    static double Percentile(double[] sortedData, double percent)
    {
        double index = (percent / 100.0) * (sortedData.Length - 1);
        int lower = (int)Math.Floor(index);
        int upper = (int)Math.Ceiling(index);

        if (lower == upper)
            return sortedData[lower];

        return sortedData[lower] +
            (index - lower) * (sortedData[upper] - sortedData[lower]);
    }
}

```

C# Outlier Detection Program

```
using System;

class OutlierProgram
{
    static void Main()
    {
        double[] data = {
            115, 182, 191, 31, 196, 1099, 5, 172, 10, 179,
            83, 21, 20, 21, 186, 177, 195, 193, 188, 199,
            62, 109, 105, 183, 110
        };

        Array.Sort(data);

        double q1 = Percentile(data, 25);
        double q3 = Percentile(data, 75);
        double iqr = q3 - q1;

        Console.WriteLine("Outliers:");

        foreach (var x in data)
        {
            if (IsOutlier(x, q1, q3, iqr))
                Console.WriteLine(x);
        }
    }

    static double Percentile(double[] sortedData, double percent)
    {
        double index = (percent / 100.0) * (sortedData.Length - 1);
        int lower = (int)Math.Floor(index);
        int upper = (int)Math.Ceiling(index);

        if (lower == upper)
            return sortedData[lower];

        return sortedData[lower] +
            (index - lower) * (sortedData[upper] - sortedData[lower]);
    }

    static bool IsOutlier(double x, double q1, double q3, double iqr)
    {
        double lowerBound = q1 - 1.5 * iqr;
        double upperBound = q3 + 1.5 * iqr;

        return x < lowerBound || x > upperBound;
    }
}
```